

A list of challenges that you faced & how did you solve them.

1.Installing HDFS & Java in Docker Containers and Sharing Configuration Files with Windows

Challenge:

Installing HDFS and Java inside Docker containers and ensuring that Hadoop configuration files could be accessed and modified from Windows through a shared folder.

Solution:

In the **Dockerfile**, I:

- Downloaded and installed all required dependencies.
- Installed **Java 17**.
- Downloaded and configured Hadoop.
- Copied the start-cluster.sh script into the container.
- Configured the container to automatically execute the script at startup so that each node starts its required Hadoop services automatically.

In docker-compose.yaml, I initially configured each node to build its image individually.

```
services:
  > Run Service
  node1:
    build: .
    container_name: node01
    hostname: node01
    ports:
      - "8481:8480"
      - "9871:9870"
      - "8081:8088"
    command: sleep infinity
    volumes:
      - ./shared:/shared
    networks:
      - cluster-network

  > Run Service
  node2:
    build: .
    container_name: node02
    hostname: node02
    ports:
      - "8482:8480"
```

I used the following commands:

```
docker compose build
```

```
docker compose up
```

This ensured:

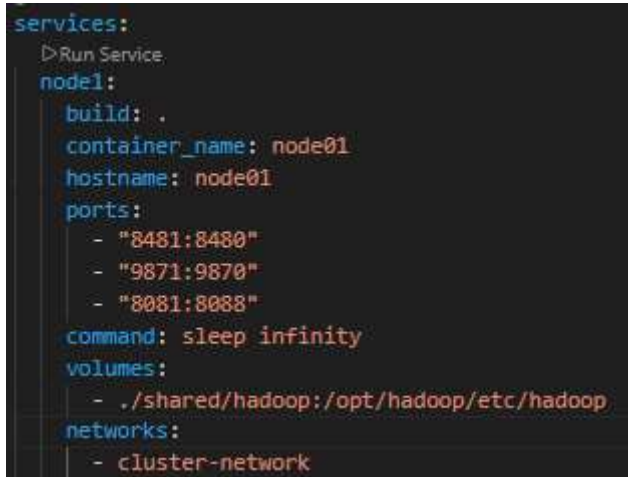
- Java and HDFS were installed automatically on all nodes.
- I verified installation using:
 - `java -version`
 - `hdfs version`

To allow configuration files to be shared between Windows and containers:

- I copied Hadoop configuration files from the container:

```
docker cp node01:/opt/hadoop/etc/hadoop ./shared/
```

And add the path of shared file in volumes to be mounted.



```
services:
  > Run Service
  node1:
    build: .
    container_name: node01
    hostname: node01
    ports:
      - "8481:8480"
      - "9871:9870"
      - "8081:8088"
    command: sleep infinity
    volumes:
      - ./shared/hadoop:/opt/hadoop/etc/hadoop
    networks:
      - cluster-network
```

Then restarted containers:

```
docker compose down
```

```
docker compose up -d
```

This enabled any changes made in the Windows shared folder to automatically reflect inside all nodes.

```

root@node01:~# ls /opt/hadoop/etc/hadoop
capacity-scheduler.xml      hadoop-user-functions.sh.example  kms-log4j.properties          ssl-client.xml.example
configuration.xml           hdfs-rbf-site.xml                kms-site.xml                  ssl-server.xml.example
container-executor.cfg      hdfs-site.xml                    log4j.properties              user_ec_policies.xml.template
core-site.xml               https-env.sh                      mapred-env.cmd                workers
hadoop-env.cmd              https-log4j.properties            mapred-env.sh                 yarn-env.cmd
hadoop-env.sh               https-site.xml                    mapred-queues.xml.template    yarn-env.sh
hadoop-metrics2.properties  kms-acls.xml                      mapred-site.xml                yarn-site.xml
hadoop-policy.xml           kms-env.sh                         mapred-site.xml                yarnservice-log4j.properties
root@node01:~#
What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug node01
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Farida> docker exec -it node01 bash
root@node01:~# ls /opt/hadoop/etc/hadoop
capacity-scheduler.xml      hadoop-user-functions.sh.example  kms-log4j.properties          ssl-client.xml.example
configuration.xml           hdfs-rbf-site.xml                kms-site.xml                  ssl-server.xml.example
container-executor.cfg      hdfs-site.xml                    log4j.properties              user_ec_policies.xml.template
core-site.xml               https-env.sh                      mapred-env.cmd                workers
hadoop-env.cmd              https-log4j.properties            mapred-env.sh                 yarn-env.cmd
hadoop-env.sh               https-site.xml                    mapred-queues.xml.template    yarn-env.sh
hadoop-metrics2.properties  kms-acls.xml                      mapred-site.xml                yarn-site.xml
hadoop-policy.xml           kms-env.sh                         mapred-site.xml                yarnservice-log4j.properties
root@node01:~#

```

2.Excessive Disk Usage Due to Multiple Image Builds

Challenge:

Because each container was building its own image, every node created a separate image of approximately **5.5 GB**.

With 5 nodes, this resulted in:

$$5.5 \text{ GB} \times 5 = 27.5 \text{ GB}$$

This storage consumption occurred even before uploading any data to HDFS, which would require additional storage.

Solution:

Since all nodes use the same dependencies and configuration, I optimized the setup:

- Only node01 builds a single image named: **hadoop-ha**
- All other nodes use this same image instead of building separate ones.

This optimization reduced total disk usage from:

$$27.5 \text{ GB} \rightarrow 5.5 \text{ GB}$$

making the system significantly more efficient.

3.NameNode High Availability – JournalNode Connection Issue

Challenge:

While testing High Availability (HA) for the NameNodes, I encountered a connectivity issue:

```
root@node02:~# hdfs haadmin -getServiceState nn2 2026-02-26 15:36:03,275 INFO ipc.Client:
Retrying connect to server: node02/172.18.0.6:8020. Already tried 0 time(s); retry policy is
RetryUpToMaximumCountWithFixedSleep(maxRetries=1, sleepTime=1000 MILLISECONDS)
Operation failed: Call From node02/172.18.0.6 to node02:8020 failed on connection exception:
java.net.ConnectException: Connection refused; For more details see:
http://wiki.apache.org/hadoop/ConnectionRefused
```

The NameNode was unable to connect to the JournalNodes.

Root Cause:

The issue was related to Docker network resolution and dynamic container IP addresses.

Solution:

- Assigned **static IP addresses** to all nodes within the Docker network.
- Added extra_hosts entries in docker-compose.yaml to explicitly define the IP addresses of the nodes.

This ensured:

- Proper hostname resolution.
- Stable communication between NameNodes and JournalNodes.
- Successful High Availability configuration.

```
networks:
  cluster-network:
    ipv4_address: 172.30.0.15
  extra_hosts:
    - "node01:172.30.0.11"
    - "node02:172.30.0.12"
    - "node03:172.30.0.13"

networks:
  cluster-network:
    driver: bridge
    ipam:
      config:
        - subnet: 172.30.0.0/16
```

4.Stoping , Starting the containers & ordering the processes

Challenge:

When the containers start to run the start-cluster.sh some processes have to start before some processes and get connection refused errors:

- DataNodes cannot start before Active NN
- Standby NN cannot bootstrap before Active NN
- HA depends on coordination

Solution:

```
wait_for_namenode() {  
    echo "Waiting for Active NameNode (nn1) to become reachable..."  
    until hdfs haadmin -getServiceState nn1 >/dev/null 2>&1; do  
        sleep 5  
    done  
    echo "Active NameNode is reachable."  
}
```

This function waits until the Active NameNode (nn1) becomes reachable.

When I stop containers and starting again the script starts a processes which is already running:

Solution:

```
start_if_not_running() {  
    local service=$1  
    local cmd=$2  
    if ! jps | grep -q "$service"; then  
        echo "Starting $service..."  
        $cmd  
    else  
        echo "$service is already running."  
    fi  
}
```

The function checks if the service is already running and starts it only if not running